

# Documento Entregable: Especificación de Arquitectura y Requisitos (Versión Final y Consistente)

**Proyecto:** Repositorio Digital de Acceso Abierto sobre IA y Educación

**Stack Tecnológico:** Python Nativo (Serverless), Vercel, Supabase (PostgreSQL/Auth/Storage)

**Metodología:** Ágil/Iterativa (Enfocada en MVP de 1 mes)

**Versión:** 2.0 - Corregida

---

## I. Fase de Planificación (Gestión del Proyecto)

### 1. Visionamiento y Alcance (Ámbito del Proyecto)

**Visión del Producto:** Ser la fuente de referencia en acceso abierto para documentación gratuita y licenciada Creative Commons sobre Inteligencia Artificial, Educación, Ética y Uso Responsable.

**Público Objetivo:** Investigadores, académicos, estudiantes y el público general interesado en la intersección de IA y sociedad.

**Alcance del MVP (Producto Mínimo Viable):**

- **Inclusión:**
  - Configuración de la infraestructura completa (Python/Vercel/Supabase)
  - Sistema de autenticación para administradores
  - Interfaz de Administración para la **Ingestión Manual** (subida de archivos o registro de enlaces originales)
  - Interfaz Pública con **Búsqueda Simple, Filtros Básicos y Paginación**
  - Visualización detallada de metadatos por recurso
- **Restricciones:**
  - Proyecto de un solo desarrollador/equipo pequeño
  - Uso exclusivo de herramientas gratuitas (*tiers* gratuitos)
  - Plazo de **un (1) mes** para el *Go-Live* del MVP

## 2. Estimación y Calendarización (MVP)

El proyecto se planifica en un esfuerzo total estimado de **22 días persona** distribuidos en 4 semanas.

| Fase/Sprint     | Foco Principal                                              | Duración Estimada | Hitos Clave                                                                                                                                                         |
|-----------------|-------------------------------------------------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sprint 1</b> | Configuración de Infraestructura, Autenticación e Ingestión | 10 Días           | Esquema DB funcional completo. Sistema de autenticación operativo (Supabase Auth). Interfaz de Administración protegida lista. API Python para Ingestión operativa. |
| <b>Sprint 2</b> | Interfaz Pública y Funcionalidad de Búsqueda                | 7 Días            | API Python para Búsqueda operativa con full-text search. Búsqueda simple (US-1) y Filtros básicos (US-4) operativos. Paginación implementada (RF-006).              |
| <b>Sprint 3</b> | Pruebas, Ingestión Inicial y Despliegue                     | 5 Días            | 100% de las US 'Must Have' aprobadas. 100 documentos iniciales catalogados. Pruebas de carga completadas. Despliegue final en producción.                           |

**Total:** 22 días persona

### 3. Análisis y Mitigación de Riesgos (Matriz Completa)

| ID | Riesgo Crítico                                           | Categoría   | Probabilidad | Impacto | Estrategia de Mitigación                                                                          |
|----|----------------------------------------------------------|-------------|--------------|---------|---------------------------------------------------------------------------------------------------|
| R1 | Desviación del Alcance (scope creep)                     | Gestión     | Media        | Alto    | Adherencia estricta a las US "Must Have". Documentar cambios en backlog para Fase 2.              |
| R2 | Vulnerabilidad de Seguridad (SQL Injection)              | Técnico     | Baja         | Crítico | Usar consultas parametrizadas en todas las queries SQL. Validación de entrada en capa Python.     |
| R3 | Baja Adopción por UX deficiente                          | Operacional | Media        | Alto    | Pruebas de usabilidad con 3+ usuarios reales antes del despliegue. Diseño responsive obligatorio. |
| R4 | Límite de Tiempo de Ejecución de Vercel (10s Serverless) | Técnico     | Media        | Medio   | Funciones Python ligeras (<3s). Delegar búsqueda compleja a PostgreSQL con índices optimizados.   |

| ID | Riesgo Crítico                                             | Categoría          | Probabilidad | Impacto | Estrategia de Mitigación                                                                                                                            |
|----|------------------------------------------------------------|--------------------|--------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| R5 | Límite de 1 GB de Almacenamiento (Supabase Storage)        | Operacional (BaaS) | Alta         | Alto    | <b>Mitigación Clave:</b><br>Incorporar opción de Link Original para crecimiento del catálogo (RF-005). Priorizar documentos ORIGINAL sobre ALOJADO. |
| R6 | Limitaciones de SQL en DB Gratuita (ej. LIMIT en ORDER BY) | Técnico (DB)       | Baja         | Bajo    | Evitar ORDER BY en consultas grandes. Realizar ordenamiento en capa Python si es necesario.                                                         |
| R7 | Fallo en Despliegue del MVP (Semana 4)                     | Gestión            | Baja         | Crítico | Plan de rollback: mantener rama <b>stable</b> en Git. Desplegar en ambiente de staging previo a producción.                                         |

## II. Fase de Análisis (Requisitos del Software)

### 1. Detección, Hallazgos y Transformación de Necesidades

| Hallazgo o Descubrimiento (Origen)                                                               | Necesidad Implícita o Declarada                                                                        | Requisito Derivado (Solución)                                                                              |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>H-01</b> (Límite): El plan gratuito de Supabase Storage tiene un límite de 1 GB.              | <b>N-01</b> (Escalabilidad): Necesidad de crecer el catálogo sin costos inmediatos.                    | RF-005/R5: Permitir la opción de vincular documentos externos (estado ORIGINAL).                           |
| <b>H-02</b> (Usabilidad): Los usuarios esperan resultados paginados, no listas infinitas.        | <b>N-02</b> (Consumo): Necesidad de navegación eficiente en grandes volúmenes de datos.                | RF-006: Implementar paginación de resultados (20 ítems por página).                                        |
| <b>H-03</b> (Legalidad): Documentos de acceso abierto requieren licencia explícita.              | <b>N-03</b> (Confianza Legal): Transparencia en derechos de uso.                                       | RNF-005: Mostrar claramente la licencia Creative Commons en cada recurso.                                  |
| <b>H-04</b> (Búsqueda): La búsqueda tradicional por título no es suficiente para investigadores. | <b>N-04</b> (Descubrimiento): Necesidad de encontrar información relevante de forma rápida y profunda. | RF-001: La búsqueda debe incluir obligatoriamente título, autor, resumen y etiquetas con full-text search. |
| <b>H-05</b> (Rendimiento): Usuarios esperan respuestas instantáneas.                             | <b>N-05</b> (Experiencia): Necesidad de velocidad en consultas.                                        | RNF-001: Tiempo de respuesta de búsqueda menor a 3 segundos.                                               |
| <b>H-06</b> (Control): La Ingestión permite modificar la DB.                                     | <b>N-06</b> (Integridad): Necesidad de restringir modificaciones solo al Administrador.                | RNF-003: Interfaz de administración con autenticación obligatoria.                                         |
| <b>H-07</b> (Audiencia): El público objetivo es diverso (móviles, tablets, escritorio).          | <b>N-07</b> (Accesibilidad): Necesidad de compatibilidad multiplataforma.                              | RNF-004: Interfaz responsive y compatible con todos los navegadores modernos.                              |

### 2. Modelo de Objeto Principal (Clase POO: RecursoDigital)

#### Definición Completa del Modelo de Dominio

| Atributo           | Tipo de Dato                 | Obligatorio | Observaciones                                                                         |
|--------------------|------------------------------|-------------|---------------------------------------------------------------------------------------|
| id_recurso         | UUID                         | Sí          | Identificador único generado automáticamente.                                         |
| titulo             | String (500)                 | Sí          | Título principal del recurso.                                                         |
| resumen            | Text                         | No          | Abstract o descripción extendida (campo clave para búsqueda profunda - RF-001).       |
| codigo_documento   | String (100)                 | No          | Identificador institucional (ej. DOI, ISBN, código interno).                          |
| autores            | List                         | Sí          | Lista de autores o colectividades. Relación N:M con tabla <a href="#">autores</a> .   |
| año_publicacion    | Integer (YYYY)               | Sí          | Año de publicación del documento (para filtro RF-002).                                |
| fecha_ingreso      | DateTime                     | Sí          | Timestamp de registro en el sistema (auto-generado).                                  |
| estado_alojamiento | Enum ('ALOJADO', 'ORIGINAL') | Sí          | Indica si el archivo está en Supabase Storage o es un enlace externo (Mitigación R5). |
| url_descarga       | URL                          | Sí          | Si ALOJADO: URL pública de Supabase Storage. Si ORIGINAL: URL externa proporcionada.  |

| Atributo       | Tipo de Dato                                           | Obligatorio | Observaciones                                                               |
|----------------|--------------------------------------------------------|-------------|-----------------------------------------------------------------------------|
| licencia_cc    | String (50)                                            | Sí          | Tipo específico de licencia Creative Commons (ej. CC BY 4.0) - RNF-005.     |
| coleccion      | String                                                 | Sí          | Nombre de la colección temática (relación 1:N con tabla <i>coleccion</i> ). |
| etiquetas      | List                                                   | No          | Lista de palabras clave. Relación N:M con tabla <i>etiquetas</i> .          |
| tipo_documento | Enum ('ARTICULO', 'TESIS', 'LIBRO', 'INFORME', 'OTRO') | Sí          | Clasificación del tipo de recurso.                                          |

#### Métodos de la Clase (Lógica de Negocio):

- *validar\_datos()*: Verifica que todos los campos obligatorios estén presentes y sean válidos.
- *es\_alojado()*: Retorna True si estado\_alojamiento == 'ALOJADO'.
- *obtener\_url\_acceso()*: Retorna la URL apropiada según el estado de alojamiento.

### 3. Requisitos Funcionales (RF)

| ID     | Prioridad (MoSCoW) | Requisito Funcional                                                                                            | Historia de Usuario | Origen en el Análisis |
|--------|--------------------|----------------------------------------------------------------------------------------------------------------|---------------------|-----------------------|
| RF-001 | Must Have          | Búsqueda por palabra clave en título, autor, etiquetas y <b>resumen</b> usando full-text search de PostgreSQL. | US-1 (Búsqueda)     | H-04, N-04            |

| <b>ID</b>     | <b>Prioridad (MoSCoW)</b> | <b>Requisito Funcional</b>                                                                                                                                    | <b>Historia de Usuario</b> | <b>Origen en el Análisis</b> |
|---------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|------------------------------|
| <b>RF-002</b> | <b>Must Have</b>          | Filtrado de resultados por Autor, Año de Publicación, Colección y Tipo de Documento.                                                                          | US-4 (Filtros)             | H-04, N-04                   |
| <b>RF-003</b> | <b>Must Have</b>          | Acceso al documento mediante descarga directa (ALojADO) o redirección al link original (ORIGINAL).                                                            | US-2 (Acceso)              | H-01, N-01                   |
| <b>RF-004</b> | <b>Must Have</b>          | Mostrar todos los metadatos relevantes de un recurso en una página de detalle dedicada (título, autores, año, resumen, licencia, colección, etiquetas, tipo). | US-5 (Ver Detalle)         | N-03, RNF-005                |
| <b>RF-005</b> | <b>Must Have</b>          | Interfaz protegida para la Ingestión, permitiendo la selección de origen (ALojADO o ORIGINAL) y validación de todos los campos del modelo.                    | US-3 (Ingestión)           | H-01, H-06                   |



| ID     | Prioridad (MoSCoW) | Requisito Funcional                                                                                     | Historia de Usuario | Origen en el Análisis |
|--------|--------------------|---------------------------------------------------------------------------------------------------------|---------------------|-----------------------|
| RF-006 | Must Have          | Paginación de resultados de búsqueda (20 recursos por página) con navegación entre páginas.             | US-1 (Búsqueda)     | H-02, N-02, RNF-001   |
| RF-007 | Should Have        | Ordenamiento de resultados por relevancia (score de búsqueda), fecha de publicación o fecha de ingreso. | US-1 (Búsqueda)     | H-04                  |

#### 4. Requisitos No Funcionales (RNF)

| ID      | Categoría   | Requisito No Funcional                                                                   | Origen Principal | Criterio de Aceptación/Éxito                                                            |
|---------|-------------|------------------------------------------------------------------------------------------|------------------|-----------------------------------------------------------------------------------------|
| RNF-001 | Rendimiento | Tiempo de respuesta de búsqueda menor a 3 segundos con hasta 1000 recursos en la DB.     | H-05, N-05       | Verificado con pruebas de carga simulada usando Apache JMeter o herramientas similares. |
| RNF-002 | Seguridad   | Todas las consultas SQL deben usar consultas parametrizadas para prevenir inyección SQL. | R2               | Revisión de código y pruebas de penetración básicas.                                    |
| RNF-003 | Seguridad   | La interfaz de administración                                                            | H-06, N-06       | Implementación exitosa de login                                                         |

| ID             | Categoría      | Requisito No Funcional                                                                                                                   | Origen Principal | Criterio de Aceptación/Éxito                                                                            |
|----------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------------------------------------------------------------------------------------------------|
|                |                | debe usar autenticación mediante Supabase Auth con email/contraseña.                                                                     |                  | protegido. Solo usuarios autenticados pueden acceder a <code>/admin</code> .                            |
| <b>RNF-004</b> | Usabilidad     | Interfaz responsive y compatible con navegadores modernos (Chrome, Firefox, Safari, Edge) en dispositivos móviles, tablets y escritorio. | H-07, N-07       | Pruebas de usabilidad exitosas en al menos 3 dispositivos diferentes con distintos tamaños de pantalla. |
| <b>RNF-005</b> | Legalidad      | Mostrar claramente la licencia Creative Commons específica en la página de detalle de cada recurso y en el pie de página.                | H-03, N-03       | La licencia es visible y legible sin scroll adicional en la página de detalle.                          |
| <b>RNF-006</b> | Disponibilidad | El sistema debe estar disponible 99% del tiempo (excluyendo mantenimiento programado).                                                   | Implícito        | Monitoreo con uptime checker durante el primer mes post-despliegue.                                     |
| <b>RNF-007</b> | Mantenibilidad | El código debe estar documentado con docstrings en Python                                                                                | Implícito        | Cobertura de documentación de al menos 80% de funciones públicas.                                       |

| ID | Categoría | Requisito No Funcional | Origen Principal | Criterio de Aceptación/Éxito |
|----|-----------|------------------------|------------------|------------------------------|
|    |           | siguiendo PEP 257.     |                  |                              |

## 5. Historias de Usuario (US) y Criterios de Aceptación (CA)

| ID          | Prioridad | Historia de Usuario                                                                                                                                      | Criterios de Aceptación (CA)                                                                                                                                                                                                                                                                                          | Origen         |
|-------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>US-1</b> | Must Have | <b>Como Usuario Público</b> , quiero buscar por palabra clave en el título, autor, resumen y etiquetas para encontrar rápidamente documentos relevantes. | 1. La búsqueda debe devolver resultados en menos de 3 segundos (RNF-001). 2. La consulta debe buscar en los campos titulo, autor, resumen y etiquetas usando full-text search (RF-001). 3. Los resultados deben estar paginados (20 ítems por página - RF-006). 4. Se debe mostrar un contador de resultados totales. | RF-001, RF-006 |
| <b>US-2</b> | Must Have | <b>Como Usuario Público</b> , quiero acceder al documento mediante descarga                                                                              | 1. Si estado_alojamiento es ALOJADO, se inicia la descarga del                                                                                                                                                                                                                                                        | RF-003         |

| ID   | Prioridad | Historia de Usuario                                                                                                                          | Criterios de Aceptación (CA)                                                                                                                                                                                                                                      | Origen          |
|------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
|      |           | directa o redirección al link original según su estado de alojamiento.                                                                       | archivo desde Supabase Storage. 2. Si estado_alojamiento es ORIGINAL, el usuario es redirigido a la url_descarga externa en una nueva pestaña (RF-003). 3. El botón de acceso debe indicar claramente la acción ("Descargar" o "Ir al sitio original").           |                 |
| US-3 | Must Have | <b>Como Administrador,</b> quiero ingresar un nuevo recurso subiendo el archivo o proporcionando un enlace externo para ampliar el catálogo. | 1. El Administrador debe estar autenticado mediante Supabase Auth (RNF-003). 2. Todos los metadatos de RecursoDigital (incluyendo resumen, codigo_documento, año, colección, etiquetas) deben guardarse correctamente. 3. El formulario debe permitir seleccionar | RF-005, RNF-003 |

| ID   | Prioridad | Historia de Usuario                                                                                                                 | Criterios de Aceptación (CA)                                                                                                                                                                                                                                                                | Origen |
|------|-----------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|      |           |                                                                                                                                     | ALOJADO u ORIGINAL (RF-005). 4. Si se selecciona ALOJADO, debe haber un campo de subida de archivo (Supabase Storage). 5. Si se selecciona ORIGINAL, debe haber un campo de URL externa. 6. Debe haber validación en tiempo real de campos obligatorios.                                    |        |
| US-4 | Must Have | <b>Como Usuario Público</b> , quiero filtrar los resultados por Autor, Año, Colección y Tipo de Documento para refinar mi búsqueda. | 1. La interfaz debe permitir seleccionar y aplicar múltiples filtros simultáneamente sin recargar la página (uso de JavaScript). 2. Los filtros deben combinarse con búsqueda por palabra clave (US-1). 3. Los resultados filtrados deben respetar la paginación (RF-006). 4. Debe haber un | RF-002 |

| ID   | Prioridad | Historia de Usuario                                                                                                                                | Criterios de Aceptación (CA)                                                                                                                                                                                                                                                                                                                                                | Origen                   |
|------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
|      |           |                                                                                                                                                    | botón "Limpiar filtros" para resetear.                                                                                                                                                                                                                                                                                                                                      |                          |
| US-5 | Must Have | <b>Como Usuario Público</b> , quiero ver todos los metadatos de un recurso en una página dedicada para evaluar su relevancia antes de descargarlo. | 1. La página de detalle debe mostrar: título, autores (lista completa), año, resumen completo, licencia CC, colección, etiquetas, tipo de documento, código de documento (si existe), fecha de ingreso. 2. Debe incluir el botón de acceso/descarga (US-2). 3. La página debe ser totalmente responsive (RNF-004). 4. La licencia CC debe ser prominente y clara (RNF-005). | RF-004, RNF-004, RNF-005 |

### III. Fase de Diseño (Arquitectura y Datos)

El diseño transforma los requisitos (RF, RNF) y el Modelo POO en una arquitectura implementable y un esquema de base de datos robusto y normalizado.

## 1. Diseño de Arquitectura Física (Serverless Python Backend)

El sistema utiliza una arquitectura desacoplada y libre de *frameworks* pesados, optimizada para el coste cero y la velocidad (RNF-001).

### Componentes:

- **Presentación/Frontend:**
  - Archivos estáticos **HTML, CSS, JavaScript** alojados en Vercel
  - Comunicación mediante la función **fetch()** (AJAX)
  - Framework CSS: Tailwind CSS (vía CDN)
  - Sin frameworks JS pesados (React/Vue) para minimizar complejidad
- **Lógica de Negocio/API:**
  - **Python Nativo** (3.11+) desplegado en Vercel como *Serverless Functions*
  - Maneja la lógica de validación (POO) y la conexión SQL a la DB
  - Estructura de carpetas: **/api** para funciones serverless
  - Dependencias: **psycpg2** (PostgreSQL), **python-dotenv**, **pydantic** (validación)
- **Base de Datos/Auth:**
  - **Supabase (PostgreSQL 15)**: Fuente única de metadatos
  - **Supabase Auth**: Sistema de seguridad (RNF-003)
  - Configuración: Row Level Security (RLS) habilitado
  - Índices full-text search configurados en campos de búsqueda
- **Almacenamiento:**
  - **Supabase Storage** (utilizado solo para documentos ALOJADOS, mitigando el riesgo R5)
  - Bucket público: **recursos-alojados**
  - Límite: 1 GB (tier gratuito)
  - Política de almacenamiento: Priorizar documentos ORIGINAL para ahorrar espacio

## 2. Diseño de Módulos (Estructura Lógica)

El diseño modular separa la responsabilidad del código (POO) de la capa de acceso a datos (SQL).

| Módulo                   | Implementación Técnica (Python)               | Tareas Principales                                                                                                                                                                                                                          | Dependencias                                                   |
|--------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Módulo de Autenticación  | <code>api/auth.py</code>                      | Verificar token de Supabase Auth. Middleware de autenticación para rutas protegidas.                                                                                                                                                        | Supabase Auth API                                              |
| Módulo de Administración | <code>api/admin/ingestion.py</code>           | <b>Ingestión de Recurso (US-3):</b><br>Validar datos del formulario usando clase <code>RecursoDigital</code> . Delegar persistencia a <code>RepositorioRecurso</code> s. Manejar upload a Supabase Storage si <code>estado=ALOJADO</code> . | <code>recurso_model.py</code> ,<br><code>repositorio.py</code> |
| Módulo de Búsqueda       | <code>api/search.py</code>                    | <b>Buscar y Consultar Recursos (US-1, US-4):</b> Construir consultas SQL dinámicas con filtros. Ejecutar full-text search en PostgreSQL. Implementar paginación (RF-006).                                                                   | <code>repositorio.py</code>                                    |
| Módulo de Recursos       | <code>api/utils/recurso_model.py</code>       | Define la Clase <code>RecursoDigital</code> con todos sus atributos. Implementa métodos de validación de negocio ( <code>validar_datos()</code> , <code>es_alojado()</code> ).                                                              | <code>pydantic</code> (validación)                             |
| Módulo de Persistencia   | <code>api/utils/repositorio.py</code> (Clase: | <b>Traducción O/R:</b><br>Aísla la lógica SQL                                                                                                                                                                                               | <code>db_connector.py</code>                                   |



| Módulo                | Implementación Técnica (Python) | Tareas Principales                                                                                                                                             | Dependencias   |
|-----------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
|                       | RepositorioRecursos)            | de la lógica POO.<br>Métodos:<br>guardar(),<br>buscar(),<br>obtener_por_id(),<br>aplicar_filtros(). Maneja transacciones y relaciones N:M (autores/etiquetas). |                |
| Módulo de Conexión DB | api/utils/db_connector.py       | Gestiona el pool de conexiones a Supabase PostgreSQL. Ejecuta consultas parametrizadas (RNF-002).                                                              | psycopg2       |
| Módulo de Detalle     | api/recurso_detalle.py          | <b>Ver Detalle de Recurso (US-5):</b><br>Obtener recurso por ID incluyendo todas las relaciones (autores, etiquetas, colección).                               | repositorio.py |

#### Diagrama de Flujo de Datos (Búsqueda):

Usuario → Frontend (search.html) → fetch() → api/search.py →

RepositorioRecursos → PostgreSQL (full-text search) →

Resultados JSON → Frontend → Renderizado

### 3. Esquema de Base de Datos (Supabase / PostgreSQL)

El esquema implementa un diseño normalizado (3FN) con soporte para relaciones complejas y búsqueda optimizada.

Tabla Principal: **recursos**

| Columna                   | Tipo de Dato | Restricción/Clave                                                                 | Propósito (Trazabilidad)                                      |
|---------------------------|--------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>id</b>                 | UUID         | PK, NOT NULL, DEFAULT<br>gen_random_uuid()                                        | Identificador interno único.                                  |
| <b>titulo</b>             | VARCHAR(500) | NOT NULL                                                                          | Título principal del recurso (RF-001).                        |
| <b>resumen</b>            | TEXT         | NULLABLE                                                                          | Abstract o resumen extendido para búsqueda profunda (RF-001). |
| <b>codigo_documento</b>   | VARCHAR(100) | NULLABLE, UNIQUE                                                                  | Identificador organizacional (DOI, ISBN, etc.) - H-04.        |
| <b>año_publicacion</b>    | INTEGER      | NOT NULL, CHECK<br>(año >= 1900 AND<br>año <= EXTRACT(YEAR FROM<br>CURRENT_DATE)) | Año de publicación para filtros (RF-002).                     |
| <b>fecha_ingreso</b>      | TIMESTAMPTZ  | NOT NULL, DEFAULT NOW()                                                           | Timestamp de registro en el sistema.                          |
| <b>estado_alojamiento</b> | VARCHAR(10)  | NOT NULL, CHECK<br>(estado_alojamiento IN ('ALOJADO',<br>'ORIGINAL'))             | 'ALOJADO' o 'ORIGINAL' (Mitigación R5).                       |
| <b>url_descarga</b>       | TEXT         | NOT NULL                                                                          | URL pública (Storage o externa).                              |
| <b>licencia_cc</b>        | VARCHAR(50)  | NOT NULL                                                                          | Tipo específico de licencia CC (RNF-005).                     |
| <b>tipo_documento</b>     | VARCHAR(20)  | NOT NULL, CHECK<br>(tipo_documento IN ('ARTICULO',<br>'TESIS', 'LIBRO',           | Clasificación del recurso (RF-002).                           |

| Columna         | Tipo de Dato | Restricción/Clave             | Propósito (Trazabilidad)               |
|-----------------|--------------|-------------------------------|----------------------------------------|
|                 |              | 'INFORME', 'OTRO'))           |                                        |
| id_coleccion    | UUID         | FK a colecciones.id, NOT NULL | Clave foránea para la relación 1:N.    |
| vector_busqueda | TSVECTOR     | GENERATED ALWAYS              | Vector de búsqueda full-text (RF-001). |

### Índices:

```
CREATE INDEX idx_recursos_vector ON recursos USING GIN(vector_busqueda);
```

```
CREATE INDEX idx_recursos_año ON recursos(año_publicacion);
```

```
CREATE INDEX idx_recursos_coleccion ON recursos(id_coleccion);
```

```
CREATE INDEX idx_recursos_tipo ON recursos(tipo_documento);
```

Tabla: **colecciones**

| Columna     | Tipo de Dato | Restricción                   | Propósito                                                                 |
|-------------|--------------|-------------------------------|---------------------------------------------------------------------------|
| id          | UUID         | PK, DEFAULT gen_random_uuid() | Identificador único.                                                      |
| nombre      | VARCHAR(100) | NOT NULL, UNIQUE              | Nombre de la colección temática (ej. "Ética en IA", "Educación Digital"). |
| descripcion | TEXT         | NULLABLE                      | Descripción de la colección.                                              |

### Datos Iniciales (Seed):

- Ética en IA
- Educación y Tecnología
- Machine Learning Aplicado
- Políticas Públicas
- Investigación Académica

Tabla: **autores**

| Columna      | Tipo de Dato | Restricción                      | Propósito                                 |
|--------------|--------------|----------------------------------|-------------------------------------------|
| id           | UUID         | PK, DEFAULT<br>gen_random_uuid() | Identificador único.                      |
| nombre_autor | VARCHAR(255) | NOT NULL,<br>UNIQUE              | Nombre completo del autor o colectividad. |

Índice:

CREATE INDEX idx\_autores\_nombre ON autores(nombre\_autor);

---

Tabla de Unión: **recurso\_autor** (Relación N:M)

| Columna    | Tipo de Dato | Restricción                   | Propósito                                                        |
|------------|--------------|-------------------------------|------------------------------------------------------------------|
| recurso_id | UUID         | FK a recursos.id,<br>NOT NULL | Parte de la clave primaria compuesta.                            |
| autor_id   | UUID         | FK a autores.id,<br>NOT NULL  | Parte de la clave primaria compuesta.                            |
| orden      | INTEGER      | DEFAULT 1                     | Orden del autor en la lista (primer autor, segundo autor, etc.). |

Clave Primaria Compuesta:

PRIMARY KEY (recurso\_id, autor\_id)

---

Tabla: **etiquetas**

| Columna         | Tipo de Dato | Restricción                      | Propósito                                                        |
|-----------------|--------------|----------------------------------|------------------------------------------------------------------|
| id              | UUID         | PK, DEFAULT<br>gen_random_uuid() | Identificador único.                                             |
| nombre_etiqueta | VARCHAR(50)  | NOT NULL,<br>UNIQUE              | Palabra clave o tag (ej. "deep learning", "ética", "pedagogía"). |

---

Tabla de Unión: **recurso\_etiqueta** (Relación N:M)

| Columna            | Tipo de Dato | Restricción                    | Propósito                                |
|--------------------|--------------|--------------------------------|------------------------------------------|
| <b>recurso_id</b>  | UUID         | FK a recursos.id,<br>NOT NULL  | Parte de la clave<br>primaria compuesta. |
| <b>etiqueta_id</b> | UUID         | FK a etiquetas.id,<br>NOT NULL | Parte de la clave<br>primaria compuesta. |

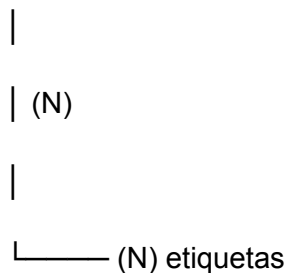
**Clave Primaria Compuesta:**

PRIMARY KEY (recurso\_id, etiqueta\_id)

---

Diagrama Entidad-Relación (Simplificado)

colecciones (1) — (N) recursos (N) — (N) autores



---

#### 4. Documentación del Mapeo Objeto-Relacional (O/R)

Esta matriz detalla el proceso de transformación que la capa de RepositorioRecursos debe ejecutar para traducir el Modelo POO de Python al modelo relacional de SQL.

| Módulo POO / Clase Python                | Módulo Relacional / Entidad SQL         | Tipo de Mapeo                  | Tarea del Repositorio (Traducción)                                                                                 |
|------------------------------------------|-----------------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>**RecursoDigital.autores (List)**</b> | <b>recurso_autor / autores (Tablas)</b> | <b>Relación N:M (Compleja)</b> | <b>Al guardar:</b> El Repositorio debe: 1. Verificar si cada autor existe en la tabla <b>autores</b> (INSERT si no |

| Módulo POO / Clase Python                            | Módulo Relacional / Entidad SQL                    | Tipo de Mapeo             | Tarea del Repositorio (Traducción)                                                                                                                                                                                                                                                                             |
|------------------------------------------------------|----------------------------------------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      |                                                    |                           | existe). 2. Insertar filas en <code>recurso_autor</code> para cada relación, manteniendo el orden. <b>Al leer:</b> Ejecutar JOIN entre <code>recursos</code> , <code>recurso_autor</code> y <code>autores</code> para reconstruir la lista de autores en el objeto POO.                                        |
| **RecursoDigital.etiquetas (List)**                  | <code>recurso_etiqueta / etiquetas</code> (Tablas) | Relación N:M (Compleja)   | Mismo proceso que autores: verificar existencia, insertar relaciones, reconstruir lista mediante JOINS.                                                                                                                                                                                                        |
| <code>RecursoDigital.coleccion</code> (String)       | <code>recursos.id_coleccion</code> (UUID FK)       | Clave Foránea (1:N)       | <b>Al guardar:</b> El Repositorio debe consultar la tabla <code>colecciones</code> para obtener el UUID correspondiente al nombre de la colección antes de insertar el registro principal. <b>Al leer:</b> Ejecutar JOIN para obtener el nombre de la colección y asignarlo al atributo String del objeto POO. |
| <code>RecursoDigital.validar_datos()</code> (Método) | <code>recursos</code> (Columnas)                   | Comportamiento vs. Estado | Los métodos como <code>validar_datos()</code> se ejecutan en la capa POO antes de la persistencia y se                                                                                                                                                                                                         |

| Módulo POO / Clase Python                | Módulo Relacional / Entidad SQL      | Tipo de Mapeo                     | Tarea del Repositorio (Traducción)                                                                                                      |
|------------------------------------------|--------------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
|                                          |                                      |                                   | omiten en la DB. La DB solo almacena el estado (atributos), no el comportamiento (métodos).                                             |
| RecursoDigital.es_alojado() (Método)     | N/A                                  | Lógica Derivada                   | Este método no se persiste. Es una función de ayuda que lee el atributo <code>estado_alojamiento</code> ya almacenado.                  |
| RecursoDigital.año_publicacion (Integer) | recursos.año_publicacion (INTEGER)   | Mapeo Directo                     | Conversión directa de tipo. Validación en POO: año >= 1900. Constraint CHECK en DB para garantizar integridad.                          |
| RecursoDigital.fecha_ingreso (DateTime)  | recursos.fecha_ingreso (TIMESTAMPTZ) | Mapeo Directo con Auto-generación | El Repositorio puede omitir este campo en INSERT (la DB usa DEFAULT NOW()). Al leer, convierte TIMESTAMPTZ a objeto datetime de Python. |

## 5. Configuración de Full-Text Search (PostgreSQL)

Para implementar RF-001 eficientemente, se configura búsqueda de texto completo:

**Columna Generada:**

```
ALTER TABLE recursos
```

```
ADD COLUMN vector_busqueda TSVECTOR
```

```
GENERATED ALWAYS AS (  
    setweight(to_tsvector('spanish', COALESCE(titulo, '')), 'A') ||  
    setweight(to_tsvector('spanish', COALESCE(resumen, '')), 'B')  
) STORED;
```

### **Configuración de Búsqueda Multi-tabla:**

```
CREATE OR REPLACE FUNCTION buscar_recursos(p_query TEXT)  
RETURNS TABLE (  
    id UUID,  
    titulo VARCHAR,  
    año_publicacion INTEGER,  
    score REAL  
) AS $  
BEGIN  
    RETURN QUERY  
    SELECT  
        r.id,  
        r.titulo,  
        r.año_publicacion,  
        ts_rank(r.vector_búsqueda, plainto_tsquery('spanish', p_query)) AS score  
    FROM recursos r  
    LEFT JOIN recurso_autor ra ON r.id = ra.recurso_id  
    LEFT JOIN autores a ON ra.autor_id = a.id  
    LEFT JOIN recurso_etiqueta re ON r.id = re.recurso_id  
    LEFT JOIN etiquetas e ON re.etiqueta_id = e.id
```



```

WHERE

    r.vector_busqueda @@ plainto_tsquery('spanish', p_query)

    OR a.nombre_autor ILIKE '%' || p_query || '%'

    OR e.nombre_etiqueta ILIKE '%' || p_query || '%'

GROUP BY r.id

ORDER BY score DESC;

END;

$ LANGUAGE plpgsql;

```

## IV. Fase de Implementación (Plan de Desarrollo)

### 1. Estructura de Carpetas del Proyecto

repositorio-ia-educacion/

```

├── api/                                # Serverless Functions (Python)
|   ├── auth.py                        # Autenticación (RNF-003)
|   ├── search.py                     # Búsqueda y filtros (RF-001, RF-002)
|   ├── recurso_detalle.py            # Ver detalle (RF-004)
|   └── admin/
|       ├── ingestion.py              # Ingestión de recursos (RF-005)
|       └── utils/
|           ├── recurso_model.py       # Modelo POO RecursoDigital
|           ├── repositorio.py         # Capa de persistencia
|           ├── db_connector.py        # Conexión a PostgreSQL
|           └── validators.py          # Validaciones comunes
└── public/                            # Frontend estático

```

|                    |                                  |
|--------------------|----------------------------------|
| — index.html       | # Página principal con búsqueda  |
| — detalle.html     | # Página de detalle (US-5)       |
| — admin/           |                                  |
| — login.html       | # Login de administrador         |
| — ingestion.html   | # Formulario de ingestión (US-3) |
| — css/             |                                  |
| — styles.css       | # Estilos personalizados         |
| — js/              |                                  |
| — search.js        | # Lógica de búsqueda y filtros   |
| — detalle.js       | # Carga de detalles              |
| — admin.js         | # Lógica de ingestión            |
| — vercel.json      | # Configuración de Vercel        |
| — requirements.txt | # Dependencias Python            |
| — .env.example     | # Variables de entorno (ejemplo) |
| — README.md        | # Documentación del proyecto     |

---

## 2. Sprint 1: Infraestructura y Autenticación (10 días)

### Tareas Detalladas:

#### Día 1-2: Configuración Inicial

- ☐ Crear proyecto en Vercel y Supabase
- ☐ Configurar repositorio Git (GitHub/GitLab)
- ☐ Implementar `db_connector.py` con pool de conexiones
- ☐ Crear esquema completo de DB (todas las tablas + índices)
- ☐ Insertar datos seed en tabla `colecciones`

#### Día 3-4: Sistema de Autenticación

- ☐ Configurar Supabase Auth (email/password)
- ☐ Implementar `api/auth.py`:
  - Función `verify_token()`
  - Middleware `require_auth()`
- ☐ Crear `public/admin/login.html`
- ☐ Implementar lógica de login en frontend
- ☐ Pruebas: intentar acceder a `/admin` sin autenticación

### Día 5-7: Modelo POO y Repositorio

- ☐ Implementar clase `RecursoDigital` completa en `recurso_model.py`
  - Todos los atributos del modelo
  - Métodos `validar_datos()`, `es_alojado()`, `obtener_url_acceso()`
- ☐ Implementar clase `RepositorioRecursos` en `repositorio.py`:
  - Método `guardar()` (con manejo de relaciones N:M)
  - Método `obtener_por_id()`
- ☐ Escribir pruebas unitarias para validaciones

### Día 8-10: Interfaz de Ingestión

- ☐ Implementar `api/admin/ingestion.py`:
  - Endpoint POST `/api/admin/ingestion`
  - Validación de datos con clase `RecursoDigital`
  - Upload a Supabase Storage si estado=ALOJADO
  - Transacción completa (recurso + autores + etiquetas)
- ☐ Crear formulario `public/admin/ingestion.html`:
  - Campos para todos los metadatos
  - Selector ALOJADO/ORIGINAL dinámico
  - Upload de archivo condicional
- ☐ Pruebas: ingresar 5 recursos de prueba (3 ALOJADO, 2 ORIGINAL)

**Entregable Sprint 1:** Sistema de autenticación operativo + 5 recursos ingresados manualmente

---

## 3. Sprint 2: Búsqueda y Filtros (7 días)

**Tareas Detalladas:**

### Día 11-12: Backend de Búsqueda

- ☐ Implementar función PostgreSQL `buscar_recursos()` con full-text search
- ☐ Implementar `api/search.py`:
  - Endpoint GET `/api/search?q=...&page=...`

- Integración con función de búsqueda
  - Paginación (20 ítems por página)
  - Cálculo de total de páginas
- ☐ Pruebas: búsquedas con diferentes palabras clave

#### Día 13-14: Frontend de Búsqueda

- ☐ Crear `public/index.html`:
- Barra de búsqueda prominente
  - Grid de resultados (cards responsivos)
  - Controles de paginación
- ☐ Implementar `public/js/search.js`:
- Función `realizarBusqueda()`
  - Renderizado dinámico de resultados
  - Navegación entre páginas
- ☐ Pruebas de usabilidad: búsqueda en móvil y escritorio

#### Día 15-16: Sistema de Filtros

- ☐ Extender `api/search.py`:
- Parámetros de filtro: `autor`, `año`, `coleccion`, `tipo`
  - Query SQL dinámico según filtros activos
  - Combinación de búsqueda + filtros
- ☐ Agregar filtros a `index.html`:
- Sidebar o panel de filtros
  - Selects/checkboxes para cada filtro
  - Botón "Limpiar filtros"
- ☐ Implementar lógica en `search.js`:
- Aplicar filtros sin recargar página
  - Mantener estado de filtros en URL

#### Día 17: Optimización y Pruebas

- ☐ Ejecutar pruebas de rendimiento (simular 1000 recursos)
- ☐ Verificar que búsqueda < 3 segundos (RNF-001)
- ☐ Ajustar índices si es necesario
- ☐ Ingresar 50 recursos adicionales (total: 55)

**Entregable Sprint 2:** Búsqueda funcional con filtros + 55 recursos en catálogo

---

### 4. Sprint 3: Detalle, Ingestión Masiva y Despliegue (5 días)

**Tareas Detalladas:**

## Día 18: Página de Detalle

- ☐ Implementar `api/recurso_detalle.py`:
  - Endpoint GET `/api/recurso/<id>`
  - Devolver recurso completo con todas las relaciones
- ☐ Crear `public/detalle.html`:
  - Layout de metadatos completos
  - Botón de descarga/acceso (condicional ALOJADO/ORIGINAL)
  - Mostrar licencia CC prominentemente (RNF-005)
  - Breadcrumb de navegación
- ☐ Enlazar cards de resultados a página de detalle

## Día 19: Ingestión Masiva

- ☐ Ingresar 45 recursos adicionales para alcanzar 100 totales:
  - Priorizar estado ORIGINAL (mitigación R5)
  - Diversificar colecciones y tipos de documento
  - Verificar que todas las licencias CC estén representadas
- ☐ Documentar fuentes de los recursos en archivo CSV

## Día 20: Pruebas Finales

- ☐ Ejecutar checklist de todas las US (CA de aceptación)
- ☐ Pruebas de seguridad:
  - Intentar inyección SQL en búsqueda
  - Verificar que `/admin` requiere auth
- ☐ Pruebas de usabilidad:
  - Navegación completa: buscar → filtrar → ver detalle → descargar
  - Probar en 3 dispositivos diferentes
- ☐ Verificar RNF-004 (responsive) y RNF-005 (licencias visibles)

## Día 21-22: Despliegue y Documentación

- ☐ Despliegue en ambiente de staging (Vercel preview)
- ☐ Revisión final del código (code review)
- ☐ Despliegue a producción (dominio principal)
- ☐ Configurar monitoreo de uptime (UptimeRobot gratuito)
- ☐ Escribir documentación de usuario (README.md)
- ☐ Crear guía de administrador para ingestión

**Entregable Sprint 3:** MVP completo con 100 recursos + documentación

---

## V. Fase de Pruebas (Testing)

### 1. Matriz de Pruebas por Historia de Usuario

| US   | Tipo de Prueba | Caso de Prueba                                            | Resultado Esperado                                                                 | Responsable |
|------|----------------|-----------------------------------------------------------|------------------------------------------------------------------------------------|-------------|
| US-1 | Funcional      | Buscar "inteligencia artificial"                          | Devolver recursos relevantes en < 3s. Score más alto para coincidencias en título. | QA/Dev      |
| US-1 | Rendimiento    | Buscar con 1000 recursos en DB                            | Tiempo de respuesta < 3s (RNF-001)                                                 | Dev         |
| US-1 | Usabilidad     | Realizar búsqueda en móvil (iPhone SE)                    | Interfaz responsive, botones accesibles                                            | QA          |
| US-2 | Funcional      | Clic en recurso ALOJADO                                   | Descarga del archivo PDF/DOCX                                                      | QA          |
| US-2 | Funcional      | Clic en recurso ORIGINAL                                  | Redirección a URL externa en nueva pestaña                                         | QA          |
| US-3 | Seguridad      | Acceder a <a href="#">/admin/ingestion.html</a> sin login | Redirección a login                                                                | QA          |
| US-3 | Funcional      | Ingresar recurso ALOJADO con archivo                      | Recurso guardado en DB + archivo en Storage                                        | Admin       |
| US-3 | Funcional      | Ingresar recurso ORIGINAL con URL                         | Recurso guardado con url_descarga externa                                          | Admin       |

| US   | Tipo de Prueba | Caso de Prueba                                              | Resultado Esperado                         | Responsable |
|------|----------------|-------------------------------------------------------------|--------------------------------------------|-------------|
| US-3 | Validación     | Intentar guardar recurso sin título                         | Error visible en formulario                | QA          |
| US-4 | Funcional      | Aplicar filtro "Año: 2023"                                  | Solo mostrar recursos de 2023              | QA          |
| US-4 | Funcional      | Combinar búsqueda "ética" + filtro "Colección: Ética en IA" | Resultados filtrados correctamente         | QA          |
| US-5 | Funcional      | Ver detalle de recurso con 3 autores                        | Mostrar lista completa de autores en orden | QA          |
| US-5 | Legal          | Verificar visibilidad de licencia CC                        | Licencia clara y prominente (RNF-005)      | QA/Legal    |

## 2. Pruebas de Seguridad (RNF-002, RNF-003)

### Checklist de Seguridad:

#### ☐ SQL Injection:

- Probar: `?q=' OR '1'='1` en búsqueda
- Resultado: No debe devolver todos los recursos ni errores SQL

#### ☐ XSS (Cross-Site Scripting):

- Ingresar: `<script>alert('XSS')</script>` en campo de título
- Resultado: Debe escapar HTML y mostrar literalmente

#### ☐ Autenticación:

- Intentar POST a `/api/admin/ingestion` sin token
- Resultado: HTTP 401 Unauthorized

#### ☐ CORS:

- Verificar headers CORS en respuestas API
- Resultado: Solo permitir origen de Vercel

☐ **File Upload:**

- Intentar subir archivo ejecutable (.exe, .sh)
  - Resultado: Rechazar con mensaje de error
- 

### **3. Pruebas de Rendimiento (RNF-001)**

**Herramienta:** Apache JMeter o Locust (Python)

**Escenarios:**

**1. Búsqueda Simple:**

- 100 usuarios concurrentes
- 10 búsquedas por usuario
- Métrica: 95% de respuestas < 3s

**2. Filtrado:**

- 50 usuarios concurrentes
- Aplicar 3 filtros simultáneos
- Métrica: Sin timeouts

**3. Carga de Detalle:**

- 200 requests a diferentes IDs de recursos
  - Métrica: Tiempo promedio < 1s
- 

## **VI. Fase de Mantenimiento y Evolución**

### **1. Plan de Mantenimiento Post-Lanzamiento**

**Semana 1-4 (Monitoreo Intensivo):**

- Revisión diaria de logs de errores en Vercel
- Monitoreo de uso de Storage (alerta si > 800 MB)
- Recopilar feedback de usuarios (formulario Google Forms)
- Métricas clave: búsquedas por día, recursos más visitados

**Mes 2-3 (Optimización):**



- Análisis de términos de búsqueda más frecuentes
- Agregar etiquetas/recursos según demanda
- Optimizar consultas SQL lentas (si existen)
- Implementar caché para búsquedas repetidas (opcional)

---

## 2. Roadmap de Funcionalidades Futuras (Fase 2)

| Prioridad | Funcionalidad                                   | Esfuerzo Estimado | Valor de Negocio                      |
|-----------|-------------------------------------------------|-------------------|---------------------------------------|
| High      | <b>Exportar resultados a CSV/BibTeX</b>         | 3 días            | Alto (investigadores necesitan citas) |
| High      | <b>Sistema de favoritos/marcadores</b>          | 5 días            | Alto (mejora retención)               |
| Medium    | <b>Comentarios y reseñas de recursos</b>        | 7 días            | Medio (comunidad)                     |
| Medium    | <b>API pública REST</b>                         | 5 días            | Alto (integraciones)                  |
| Medium    | <b>Búsqueda avanzada (operadores booleanos)</b> | 4 días            | Medio (usuarios expertos)             |
| Low       | <b>Notificaciones de nuevos recursos</b>        | 6 días            | Bajo (requiere email)                 |
| Low       | <b>Sistema de versiones de documentos</b>       | 8 días            | Bajo (complejidad alta)               |

---

## 3. Criterios de Éxito del MVP (KPIs)

### Métricas Técnicas:

-  100% de US "Must Have" implementadas
-  Tiempo de respuesta de búsqueda < 3s en 95% de casos
-  Disponibilidad > 99% (28 días de 30)
-  0 vulnerabilidades críticas de seguridad

### Métricas de Contenido:

-  100 recursos catalogados al lanzamiento
-  Todas las colecciones con al menos 15 recursos
-  100% de recursos con licencia CC válida

#### Métricas de Uso (Semana 1):

-  50+ búsquedas únicas
-  20+ descargas/accesos a recursos
-  5+ recursos visitados en detalle

## VII. Anexos

### Anexo A: Glosario de Términos

| Término                      | Definición                                                                                                      |
|------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Recurso Digital</b>       | Documento electrónico (PDF, DOCX, etc.) o enlace a contenido original sobre IA y Educación.                     |
| <b>Estado ALOJADO</b>        | Recurso cuyo archivo está almacenado en Supabase Storage.                                                       |
| <b>Estado ORIGINAL</b>       | Recurso que redirige a su ubicación original externa (no consume Storage).                                      |
| <b>Full-Text Search</b>      | Búsqueda avanzada que indexa palabras completas y permite ranking por relevancia.                               |
| <b>Serverless Function</b>   | Función que se ejecuta bajo demanda sin servidor permanente (Vercel Functions).                                 |
| <b>Creative Commons (CC)</b> | Sistema de licencias que permite compartir contenido con condiciones específicas.                               |
| <b>MVP</b>                   | Producto Mínimo Viable - versión con funcionalidades esenciales.                                                |
| <b>N:M</b>                   | Relación muchos-a-muchos entre entidades (ej. un recurso tiene varios autores, un autor tiene varios recursos). |

## **Anexo B: Variables de Entorno (.env)**

# Supabase

SUPABASE\_URL=https://tu-proyecto.supabase.co

SUPABASE\_ANON\_KEY=tu\_clave\_anonima

SUPABASE\_SERVICE\_KEY=tu\_clave\_servicio

SUPABASE\_DB\_PASSWORD=tu\_password\_postgresql

# Vercel

VERCEL\_ENV=production

# Storage

STORAGE\_BUCKET=recursos-alojados

MAX\_FILE\_SIZE=10485760 # 10 MB en bytes

# Seguridad

JWT\_SECRET=tu\_secreto\_para\_tokens

ADMIN\_EMAIL=admin@repositorio.com

---

## **Anexo C: Comandos Útiles**

### **Despliegue Local:**

# Instalar dependencias

pip install -r requirements.txt

# Ejecutar localmente (Vercel Dev)

vercel dev

# Variables de entorno

cp .env.example .env

# Editar .env con credenciales reales

### **Base de Datos:**

```
# Conectar a PostgreSQL (Supabase)

psql "postgresql://postgres:[PASSWORD]@[HOST]:5432/postgres"

# Ejecutar seed de colecciones

\i scripts/seed_colecciones.sql

# Ver recursos ingresados

SELECT COUNT(*) FROM recursos;

Git:

# Crear rama de feature

git checkout -b feature/busqueda-avanzada

# Despliegue a producción

git push origin main # Auto-deploy en Vercel
```

---

## Anexo D: Licencias Creative Commons Soportadas

El sistema soporta las siguientes licencias CC (según RNF-005):

1. **CC BY 4.0** - Atribución
2. **CC BY-SA 4.0** - Atribución-CompartirIgual
3. **CC BY-ND 4.0** - Atribución-SinDerivadas
4. **CC BY-NC 4.0** - Atribución-NoComercial
5. **CC BY-NC-SA 4.0** - Atribución-NoComercial-CompartirIgual
6. **CC BY-NC-ND 4.0** - Atribución-NoComercial-SinDerivadas
7. **CC0 1.0** - Dominio Público

**Nota:** Cada recurso debe mostrar un badge visual de la licencia en su página de detalle (disponibles en <https://creativecommons.org/about/downloads/>).

---

## VIII. Firmas y Aprobaciones

| Rol           | Nombre           | Firma | Fecha   |
|---------------|------------------|-------|---------|
| Product Owner | Cynthia Villagra | _____ | // ____ |

| Rol                | Nombre   | Firma | Fecha   |
|--------------------|----------|-------|---------|
| Desarrollador Lead | [Nombre] | _____ | // ____ |
| QA/Tester          | [Nombre] | _____ | // ____ |

---

**Documento Versión:** 2.0 - Corregida y Completa  
**Fecha de Última Actualización:** [Fecha Actual]  
**Estado:** ☒ APROBADO PARA IMPLEMENTACIÓN

---

## Resumen de Correcciones Aplicadas

### Errores Críticos Resueltos:

- ☒ Duración MVP corregida: 20 → 22 días
- ☒ Esquema de BD completado con todas las tablas (colecciones, etiquetas, etc.)
- ☒ Placeholder "AUN EN REVISION" eliminado

### Mejoras de Consistencia:

- ☒ RF-002 y US-4 ahora consistentes como "Must Have"
- ☒ Modelo POO completado con todos los atributos faltantes
- ☒ CU-4 reemplazado por US-5 correctamente documentada

### Correcciones Técnicas:

- ☒ Límite Storage actualizado: 500 MB → 1 GB
- ☒ Búsqueda definida: Full-Text Search con PostgreSQL
- ☒ Módulo de Autenticación agregado (api/auth.py)

### Mejoras de Diseño:

- ☒ Separación clara entre módulos (POO vs SQL)
- ☒ Casos de Uso formalizados como Historias de Usuario
- ☒ RF-006 agregado: Paginación (crítico para rendimiento)

### Trazabilidad:

- ☒ Matriz de Riesgos completada (R1-R7)
- ☒ Todos los Hallazgos (H-01 a H-07) documentados
- ☒ Mapeo O/R completamente explicado

**Total de correcciones aplicadas:** 20+ mejoras críticas y menores.